



LeetCode 373: Find K Pairs with Smallest Sums

1. Problem Title & Link

Title: LeetCode 373: Find K Pairs with Smallest Sums

Link: <https://leetcode.com/problems/find-k-pairs-with-smallest-sums/>

2. Problem Statement (Short Summary)

You are given two sorted arrays:

nums1

nums2

and an integer:

k

A pair is:

(nums1[i], nums2[j])

Return the k pairs having the smallest sums.

3. Examples (Input → Output)

Example 1

Input:

nums1 = [1,7,11]

nums2 = [2,4,6]

k = 3

Output:

[[1,2],[1,4],[1,6]]

Pair sums:

(1,2)=3

(1,4)=5

(1,6)=7

(7,2)=9

(7,4)=11

...

Smallest 3 pairs are returned.

Example 2

Input:

nums1 = [1,1,2]

nums2 = [1,2,3]

k = 2



Output:

[[1,1],[1,1]]

Example 3

Input:

nums1 = [1,2]

nums2 = [3]

k = 3

Output:

[[1,3],[2,3]]

Only two pairs exist.

4. Constraints

- $1 \leq \text{nums1.length} \leq 10^5$
- $1 \leq \text{nums2.length} \leq 10^5$
- Both arrays are sorted
- $1 \leq k \leq 10^4$

Need better than:

$O(n \times m)$

5. Core Concept (Pattern / Topic)

Min Heap (Priority Queue)

Related Topics:

- K-Way Merge
- Heap
- Sorted Matrix
- Best First Search

6. Thought Process (Step-by-Step Explanation)

Brute Force

Generate all pairs.

Example:

nums1=[1,7,11]

nums2=[2,4,6]

Generate:

(1,2)

(1,4)



(1,6)

(7,2)

(7,4)

(7,6)

(11,2)

(11,4)

(11,6)

Sort all pairs.

Complexity:

$O(n*m \log(n*m))$

Too expensive.

Key Observation

Arrays are already sorted.

If:

(1,2)

is the smallest pair,

then next candidates are:

(1,4)

(7,2)

We don't need all pairs.

We only need the next smallest candidate each time.

This suggests:

Min Heap

Heap Strategy

Initially insert:

(1,2)

(7,2)

(11,2)

using:

(sum, i, j)

Heap:

3 -> (1,2)



9 -> (7,2)

13 -> (11,2)

Remove smallest.

When:

(1,2)

is removed,

insert:

(1,4)

because it is the next pair in that row.

Why It Works

Each row:

$\text{nums1}[i] + \text{nums2}[0]$

$\text{nums1}[i] + \text{nums2}[1]$

$\text{nums1}[i] + \text{nums2}[2]$

...

is sorted.

The heap efficiently merges these sorted sequences.

Exactly like:

Merge K Sorted Lists

7. Visual / Intuition Diagram (ASCII Diagram)

```

Pair Sum Matrix:
      2   4   6
1      3   5   7
7      9  11  13
11     13  15  17
Start:
3
9
13
Pop:
3
Push:
5
Heap:
5
9
13

```



Continue until k pairs collected.

8. Pseudocode (Language Independent)

```
Create min heap

Push:

(i,0)
for first min(k,n) rows

answer = []

while heap not empty
    and k > 0

    pop smallest pair

    add pair to answer

    push next column

return answer
```

9. Code Implementation

✓ Python

```
import heapq

class Solution:

    def kSmallestPairs(
        self,
        nums1,
        nums2,
        k
    ):

        if not nums1 or not nums2:
            return []

        heap = []

        for i in range(
            min(k, len(nums1))
        ):

            heapq.heappush(
```



```

        heap,
        (
            nums1[i] + nums2[0],
            i,
            0
        )
    )

answer = []

while heap and k:

    _, i, j = heapq.heappop(heap)

    answer.append(
        [nums1[i], nums2[j]]
    )

    if j + 1 < len(nums2):

        heapq.heappush(
            heap,
            (
                nums1[i]
                + nums2[j + 1],
                i,
                j + 1
            )
        )

    k -= 1

return answer

```

✓ Java

```

import java.util.*;

class Solution {

    public List<List<Integer>>
    kSmallestPairs(
        int[] nums1,
        int[] nums2,
        int k
    ) {

        List<List<Integer>>
        answer = new ArrayList<>();
    }
}

```



```
if (nums1.length == 0 ||
    nums2.length == 0)
    return answer;

PriorityQueue<int[]> pq =
    new PriorityQueue<>(
        (a,b) -> a[0]-b[0]
    );

for (
    int i = 0;
    i < Math.min(k, nums1.length);
    i++
) {

    pq.offer(
        new int[]{
            nums1[i]+nums2[0],
            i,
            0
        }
    );
}

while (
    !pq.isEmpty() &&
    k-- > 0
) {

    int[] curr = pq.poll();

    int i = curr[1];
    int j = curr[2];

    answer.add(
        Arrays.asList(
            nums1[i],
            nums2[j]
        )
    );

    if (
        j + 1 <
        nums2.length
    ) {

        pq.offer(
            new int[]{
                nums1[i]
                + nums2[j+1],
```



```

        i,
        j+1
    }
    );
}
}
return answer;
}
}

```

10. Time & Space Complexity

Time Complexity

Heap size:

$\min(k, n)$

Operations:

k heap operations

Total:

$O(k \log(\min(k, n)))$

Space Complexity

$O(\min(k, n))$

For heap.

11. Common Mistakes / Edge Cases

Mistake 1

Generating all pairs.

$O(n * m)$

Memory explosion.

Mistake 2

Not checking:

$j+1 < \text{len}(\text{nums2})$

before pushing.

Mistake 3

Using max heap instead of min heap.

12. Detailed Dry Run (Step-by-Step Table)



Input:

nums1=[1,7,11]

nums2=[2,4,6]

k=3

Initial Heap:

Sum	Pair
3	(1,2)
9	(7,2)
13	(11,2)

Pop:

(1,2)

Answer:

[[1,2]]

Push:

(1,4)

Heap:

Sum	Pair
5	(1,4)
9	(7,2)
13	(11,2)

Pop:

(1,4)

Answer:

[[1,2],[1,4]]

Push:

(1,6)

Pop:

(1,6)

Answer:

[[1,2],[1,4],[1,6]]

Done.



13. Common Use Cases (Real-Life / Interview)

Recommendation Systems

Find top-k combinations.

Search Engines

Merge ranked result lists.

E-commerce

Top-k cheapest product bundles.

Analytics

Find smallest pair combinations across datasets.

14. Common Traps (Important!)

- Generating all pairs
- Using sorting after pair generation
- Forgetting heap initialization
- Pushing invalid indices
- Using wrong heap ordering

15. Builds To (Related LeetCode Problems)

1. LC 23 — Merge K Sorted Lists
2. LC 378 — Kth Smallest Element in a Sorted Matrix
3. LC 703 — Kth Largest Element in a Stream
4. LC 215 — Kth Largest Element in an Array
5. LC 373 — Find K Pairs with Smallest Sums

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Generate All Pairs	$O(nm \log(nm))$	$O(nm)$	Very Slow
Sorting All Pairs	$O(nm \log(nm))$	$O(nm)$	Memory Heavy
Min Heap	$O(k \log(\min(k, n)))$	$O(\min(k, n))$	Optimal

17. Why This Solution Works (Short Intuition)

Each row of pair sums is already sorted because the arrays are sorted. A min heap helps efficiently merge these sorted rows and always gives the next smallest pair without generating all possible combinations.



18. Variations / Follow-Up Questions

Follow-Up 1

What if we need:

K Largest Pairs

Use a Max Heap.

Follow-Up 2

What if arrays are not sorted?

Sort first, then apply the same approach.

Follow-Up 3

What if we only need:

Kth Smallest Pair

Use Binary Search on Answer or Heap-based extraction.